

University Course Registration System

Name	ID
Leander : Rawan Abdullah Al-Mahdawi	44610911
Lama Hassan Al-Mahdawi	44610371
Dana Nayef Al-Mahabi	44610881
Dima Hashim Al-Saab	44611454



Table Contents

#	Title	Page
1	Introduction	3
2	Proposal	3
3	Planning	4
4	Requirements	5
5	Design	6
6	Implementation	10
7	Testing	11
8	Deployment	13
9	Conclusion	16

Introduction

This report documents our work for the SE1201 Foundations of Software Engineering project. We were required to build a software system following the System Development Life Cycle working as a team of four members

Our project is a University Course Registration System. It is a web application that allows students to browse and enroll in courses view their weekly schedule and drop courses. It also provides administrators with tools to manage the course catalog and student accounts

This report covers all seven phases of the project: the proposal, planning, requirements, design implementation, testing, and deployment. Each section explains what we did in that phase and how we applied the concepts learned in the course

1. Proposal

Our project is a University Course Registration System. The problem we wanted to solve is the difficulty students face during registration each semester. They often do not know which courses are available, which are full, or if they have schedule clashes. They may also try to enroll in a course without completing the prerequisite. On the admin side, managing courses and student records manually is time consuming

Our solution is a web based system. Students log in and see all available courses with details like schedule, credit hours, remaining seats, and prerequisite. When they enroll, the system checks automatically for duplicates, capacity, prerequisite, and schedule conflicts. If all checks pass, the enrollment goes through. Students can view their weekly schedule and drop courses. Admins have their own login and can add, edit, and delete courses. They can also view all students, see what each one is enrolled in, add new students, and delete students

This system makes registration faster for students and administration easier for staff

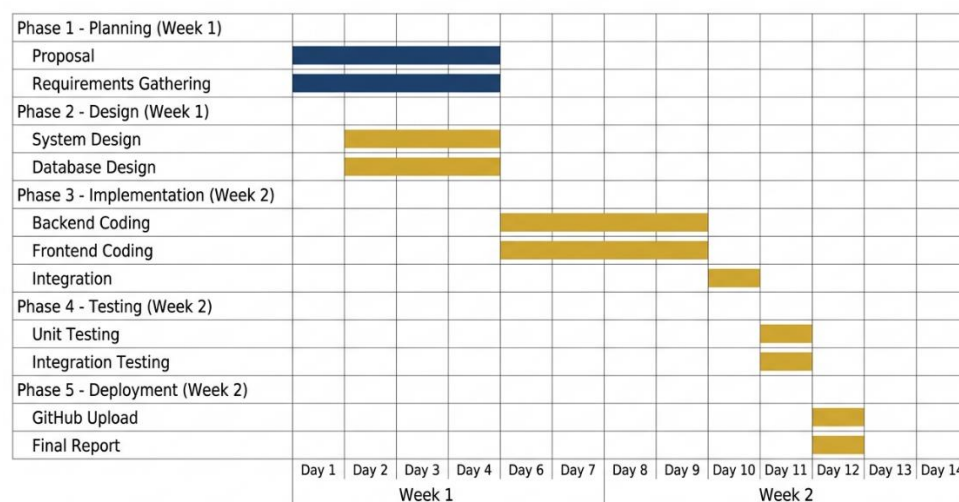
2. Planning

We planned the project to be completed over 2 weeks. We divided the work into phases to follow the software development life cycle we studied in the course.

The first two weeks were for the proposal and gathering requirements. We discussed what the system needed to do and wrote down all the features. Weeks three and four were for design. During this time we drew all the diagrams, planned the database tables, and decided on the project structure. Weeks five to eight were the longest phase, which was the implementation and coding. We built the backend with Flask, created the HTML templates, wrote the CSS, and connected everything together. Weeks nine and ten were for testing. We applied the four testing strategies we learned in the course. The last two weeks were for deployment, uploading the project to GitHub writing the final report, and preparing the submission files

We also created a Gantt chart to visualize this timeline. The chart is included in the report

SE1201 Project Gantt Chart



The work was divided among the four team members. I took the role of team leader and handled the backend development, including the Flask routes, authentication, and business logic. The second member worked on the frontend, which includes the HTML pages, CSS styling, and JavaScript validation. The third member designed the database models and handled most of the testThe fourth member was responsible for documentation, meeting logs and deployment to GitHub

We held regular meetings throughout the project. We documented each meeting in an Excel sheet named Meeting_logs.xlsx. In each meeting we recorded who attended, what we discussed, the results, and the date of the next meeting. Over the 12 weeks we held six meetings in total. The first was to choose the project topic. The second was for requirements gathering. The third was to review the design diagrams. The fourth was to check on implementation progress. The fifth was for testing and bug fixing. The sixth was the final review before submitss

3. Requirements

We identified the requirements for the system by thinking about what both types of users need to do. We split them into functional and non-functional requirements.

Functional Requirements:

These describe what the system actually does

For students, the system should let them log in using their university ID and password. After logging in, they should see a dashboard with a summary of their registered courses and total credit hours. They should be able to browse all available courses. Each course should show its code, name, credit hours, schedule, how many seats are left, and whether there is a prerequisite. When a student clicks to enroll, the system must check several things. It must make sure the student is not already enrolled in that course. It must check that there are still seats available. It must verify that the student has completed the prerequisite if one is required. It must check that the new course does not clash in time with another course the student is already taking. If all checks pass, the enrollment is successful and the seat count goes down by one. Students should also be able to view their weekly schedule in a calendar format and drop courses they no longer want.

For administrators, the system should let them log in with a username and password. They should see a dashboard with statistics showing the total number of students, courses, and enrollments. They should be able to manage courses, meaning they can add new courses, edit existing course details, and delete courses that have no students enrolled. They should be able to view a list of all students with their details and see which courses each student is taking. They should also be able to add new student accounts manually and delete student accounts when needed.

Non-Functional Requirements:

These describe how the system should behave in terms of quality and security.

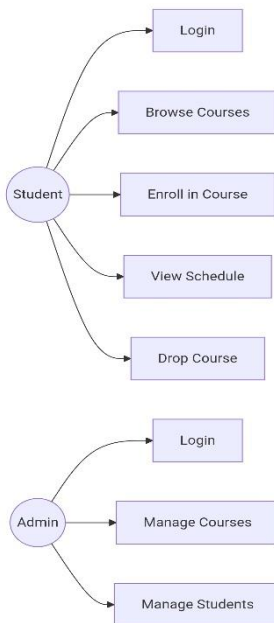
The system must be secure. All passwords must be hashed, so even if someone accesses the database they cannot see the real passwords. We used Werkzeug with the pbkdf2:sha256 algorithm for this. All forms must be protected against CSRF attacks, and we used Flask-WTF for this purpose. SQL injection must be prevented, and since we used SQLAlchemy ORM for all database operations, this is covered. Any data shown on the screen must be safe from XSS attacks, and Jinja2 templates handle this by automatically escaping output. Users who are not logged in should not be able to access any protected pages. If they try, they get redirected to the login page with an error message.\

The system should be easy to use. The interface should be clean and should work on different screen sizes, including desktops, tablets, and mobile phones. Error messages should be clear so the user understands what went wrong. Success messages should appear when something is done correctly. Pages should load quickly, within two to three seconds under normal conditions

4. Design

During the design phase, we created several diagrams to plan the system before writing any code. These diagrams were introduced in the SE1201 course and helped us understand how the system should work

Case Diagram:



This diagram shows the two types of users and what they can do in the system. The Student actor can perform five actions:

login, browse courses,

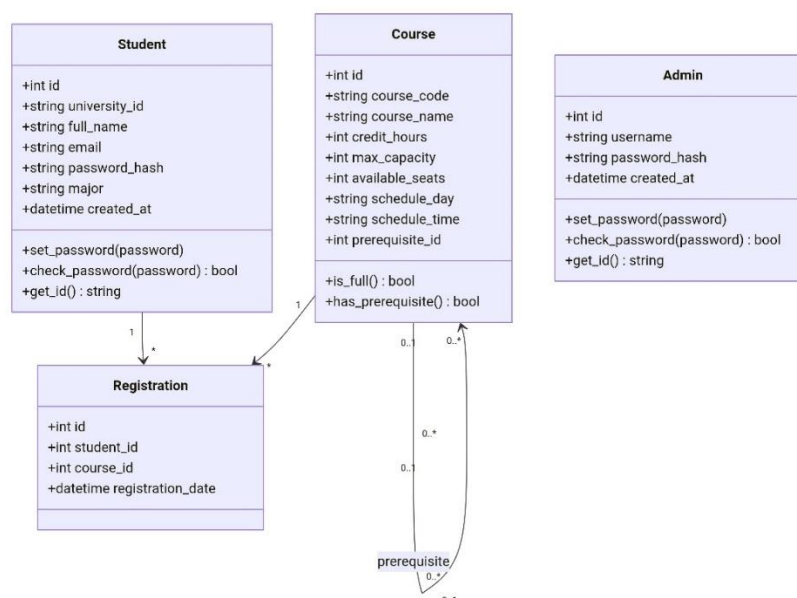
enroll in a course, view their schedule, and drop a course.

The Admin actor can perform three main actions: login, manage courses, and manage students. Manage courses includes adding, editing, and deleting courses.

Manage students includes viewing all students, adding new ones, and deleting them

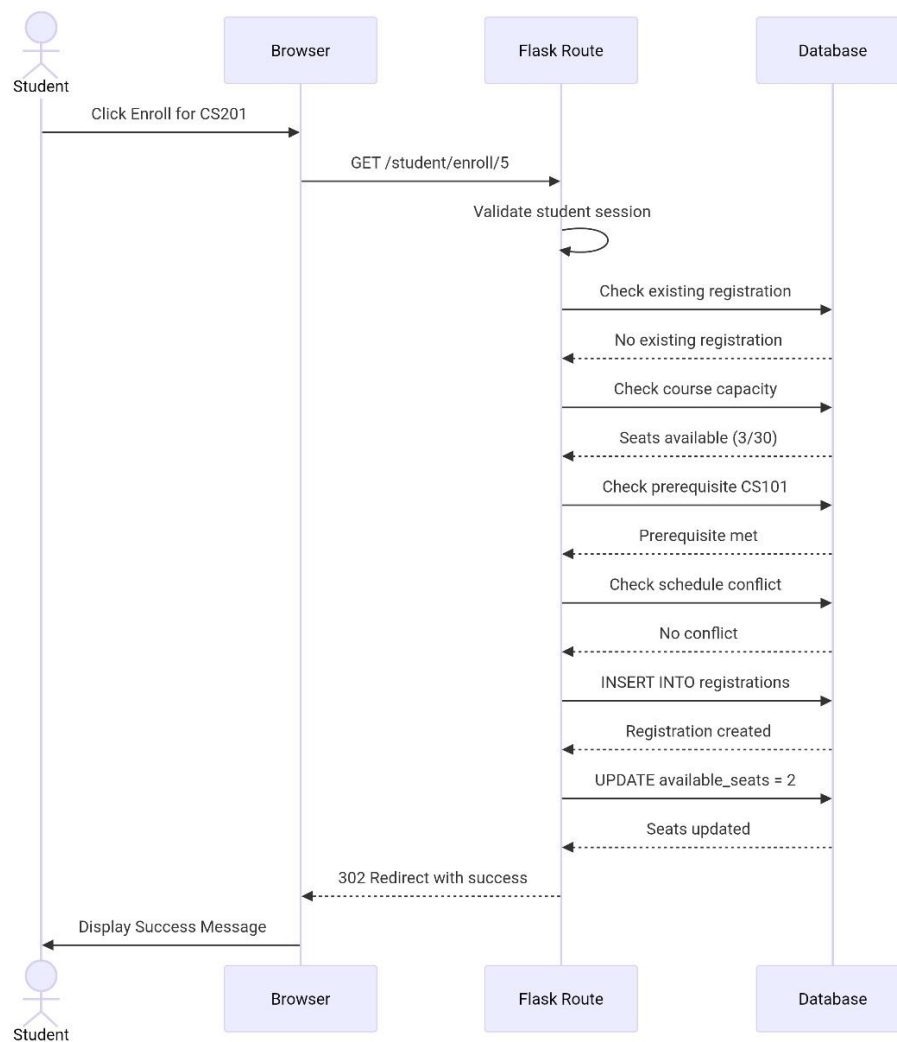
Class Diagram:

This diagram shows the four main classes in our code and how they relate to each other. The Student class has attributes like `university_id`, `full_name`, `email`, `password_hash`, and `major`. The Course class has `course_code`, `course_name`, `credit_hours`, `max_capacity`, `available_seats`, `schedule_day`, `schedule_time`, and `prerequisite_id`. The Registration class connects students to courses and has `student_id`, `course_id`, and `registration_date`. The Admin class has `username` and `password_hash`. The relationships are: one student can have many registrations, one course can have many registrations, and a course can reference another course as its prerequisite



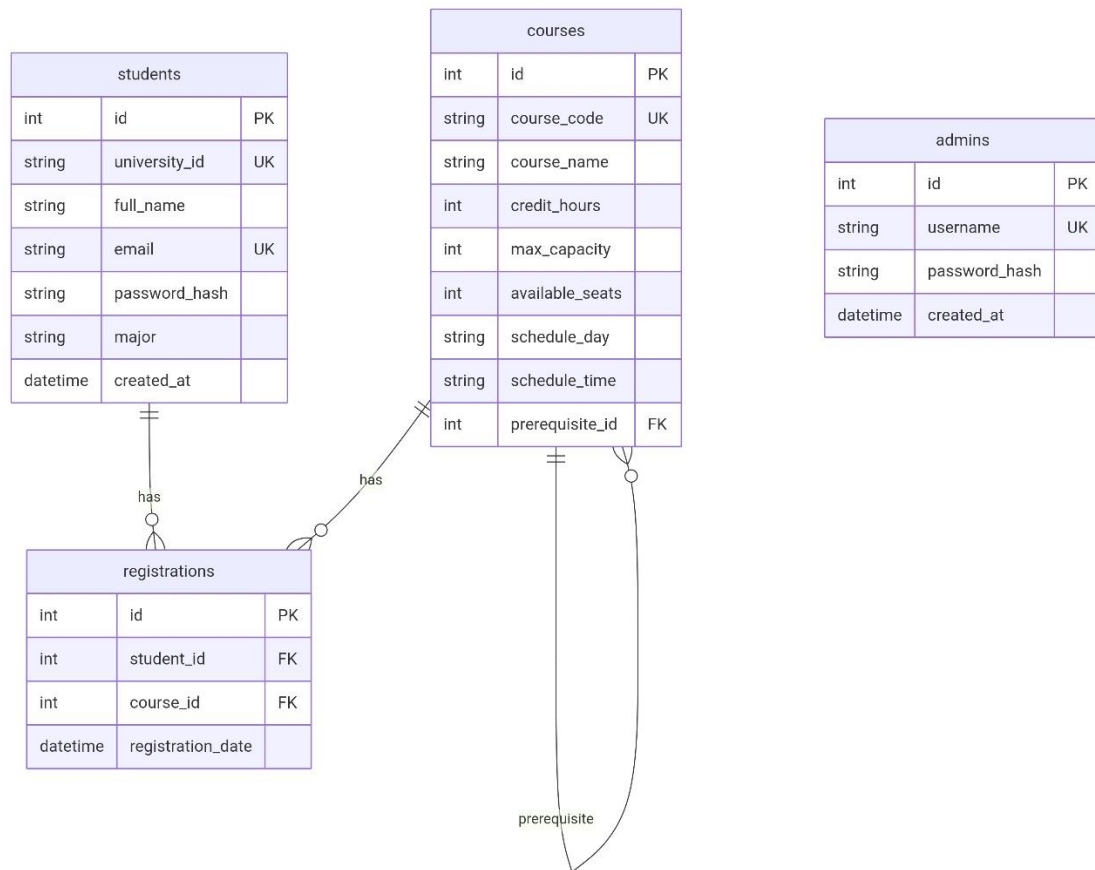
Sequence Diagram:

We drew a sequence diagram to show the enrollment process step by step. It starts with the student clicking the enroll button in the browser. The browser sends a request to the Flask route. The route checks with the database whether the student is already registered, whether seats are available, whether the prerequisite is met, and whether there is a schedule conflict. If all checks pass, the route creates a registration record and updates the available seats. Then it redirects the browser back to the courses page with a success message.



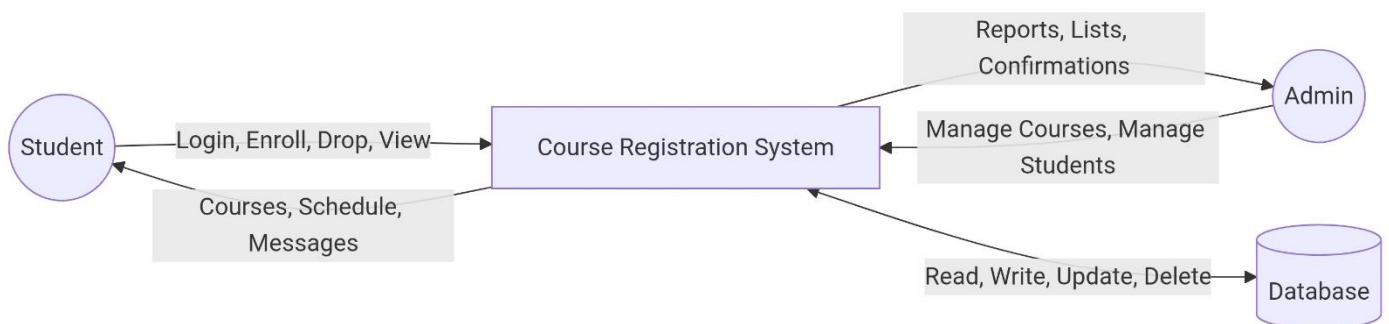
Entity-Relationship Diagram:

This diagram shows the database tables and their connections. The students table has a one-to-many relationship with the registrations table. The courses table also has a one-to-many relationship with registrations. The courses table has a self-referencing relationship for the prerequisite. The admins table is independent.



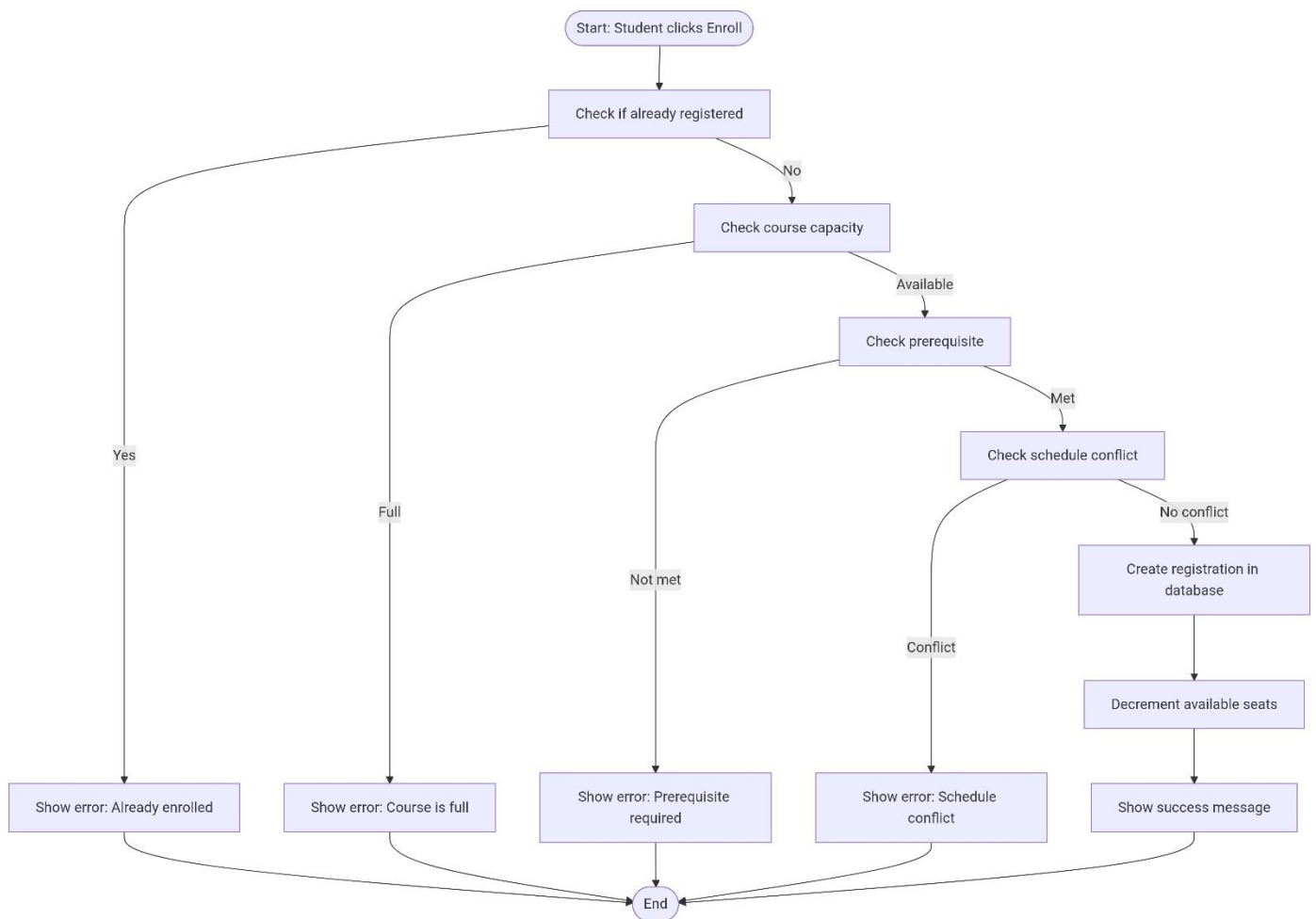
Data Flow Diagram:

We drew a level 0 DFD to show how data flows in the system. The Student sends data like login credentials and enrollment requests to the system and receives data like course lists and schedule information. The Admin sends course and student data to the system and receives reports and confirmation messages. The system reads from and writes to the database.



Activity Diagram:

This diagram shows the flow of the enrollment process. It starts when the student clicks enroll. Then it goes through each check one by one. If the student is already registered, it shows an error and stops. If the course is full, it shows an error and stops. If the prerequisite is not met, it shows an error and stops. If there is a schedule conflict, it shows an error and stops. If all checks pass, it creates the registration, updates the seats, and shows a success message.



5. Implementation

We built the system using several technologies. For the backend, we used Python with the Flask framework. Flask is lightweight and was suitable for our project size. For the database, we used SQLite with SQLAlchemy as the ORM. SQLAlchemy allowed us to write all database operations in Python code instead of raw SQL, which helps prevent SQL injection. We used Flask-Login to handle user sessions and keep users logged in. We used Flask-WTF with WTForms for all our forms, which gave us built-in CSRF protection. For password security, we used Werkzeug's password hashing functions. For the frontend, we wrote everything from scratch using HTML, CSS, and vanilla JavaScript. We did not use any frontend frameworks like Bootstrap so we could have full control over the design

The project is organized into clear modules. Here is the folder structure:

The main folder contains `app.py` which is the entry point that runs the whole application. `config.py` stores all settings including the secret key and database location. `models.py` defines our four database tables as Python classes. `forms.py` contains our WTForms classes for login and course management

Inside the `app` folder, we have `decorators.py` for access control. It has three decorators: `student_required` makes sure only logged-in students can access certain routes, `admin_required` does the same for admins, and `logout_required` redirects already logged-in users away from the login page. The `routes` folder splits the routes by user type. `auth.py` handles login and logout for both students and admins. `student.py` handles student-specific routes like the dashboard, browsing courses, enrolling, viewing the schedule, and dropping courses. `admin.py` handles admin routes like the dashboard, managing courses with full CRUD operations, viewing students, adding students, and deleting students

The `templates` folder contains all our HTML files organized by user type. `base.html` is the main layout that all other pages inherit from. It has the navigation bar, flash messages area, and footer. `index.html` is the login page. The `student` folder has `dashboard.html` for the student home page, `courses.html` for browsing and enrolling in courses, and `schedule.html` for viewing the weekly schedule. The `admin` folder has `dashboard.html` for the admin home page, `courses.html` for managing courses, and `students.html` for viewing and managing students

The `static` folder contains `style.css` for all our custom styling and `main.js` for client-side JavaScript like form validation, confirmation dialogs, and auto-hiding flash messages

Every Python function we wrote has a docstring explaining what it does, what parameters it takes, and what it returns. Each file starts with a comment block that explains the purpose of the file and which module it belongs to. We followed consistent naming conventions. Python uses snake_case and JavaScript uses camelCase

Here are some important parts of our code. The enrollment function in student.py is a good example of how the system works. Before inserting anything into the database, it runs four checks in sequence. It queries the database to see if the student is already registered for that course. It checks the course's available_seats field to see if it is greater than zero. It checks if the course has a prerequisite and if so, it verifies the student has completed it by looking at their existing registrations. It checks for schedule conflicts by comparing the new course day and time with all the student's current courses. If any check fails, the function returns an error message and stops. Only if all four pass does it create the registration and decrease the seat count by one.

Another important part is the password handling in models.py. We never store raw passwords. When creating a user, we call set_password which uses generate_password_hash. When a user logs in, we call check_password which uses check_password_hash. These functions handle all the cryptography for us using a secure algorithm.

The decorators in decorators.py protect our routes. For example, the student_required decorator first checks if the user is authenticated at all. If not, it redirects to the login page. Then it checks the class name of the current user to make sure it is Student and not Admin. If someone tries to access a student route with an admin account, they get an error and are redirected. This makes sure each user type only sees what they are supposed to see

6. Testing

We applied the four testing strategies that were taught in the SE1201 course. Testing was an important part of making sure our system works correctly and handles errors properly

Unit Testing:

We tested individual functions to make sure each one does what it is supposed to do on its own. We tested the password functions in the Student and Admin models. We gave set_password a password like "Student@123" and made sure it produced a long hashed string that looks nothing like the original. Then we tested check_password with the correct password and it returned True. With a wrong password it returned False. We also tested the helper functions used during enrollment. We tested check_schedule_conflict and gave it a course at the same day and time as an existing registration, and it correctly returned True. When we gave it a course at a different time, it returned False. We tested check_prerequisite_met with a course that has no prerequisite and it returned True. With a course that has a prerequisite the student has completed, it returned True. With a prerequisite not completed, it returned False. We tested the is_full function on the Course model. When available_seats was zero, it returned True. When seats were still available, it returned False. Every unit we tested worked as expected

Integration Testing:

Integration testing means checking that different parts of the system work together correctly. We tested the full flow from start to finish for the most important actions. We tested logging in as a student with valid credentials. The session was created and the browser was redirected to the student dashboard. We tested logging in with wrong credentials and the system showed an error message without crashing. We tested the enrollment process end to end. When a student enrolled in a course, the registration record appeared in the database and the `available_seats` count decreased by one. We tested trying to enroll in the same course twice and the system blocked it with a clear message. We tested enrolling in a course that is full and the system said the course is full. We tested enrolling in a course where the student has not met the prerequisite and the system said which prerequisite is needed. We tested enrolling in a course that clashes in time with another course and the system reported the conflict. We tested dropping a course. The registration was deleted from the database and the `available_seats` count went back up by one. On the admin side, we tested adding a new course. It appeared in the course table immediately. We tested editing a course and the changes were saved. We tested deleting a course with no students enrolled and it was removed. We tested trying to delete a course that has students in it and the system refused with a message saying how many students are enrolled. We tested adding a new student and the account was created. We tested deleting a student and the account was removed along with all their registrations, and the seats were

Static Testing:

Static testing means reviewing the code without running it. We carefully looked through all our Python files. We checked for PEP 8 style issues like proper indentation, line length, and spacing. Everything looked consistent. We checked for any raw SQL strings in the code and found none. All database operations use SQLAlchemy ORM methods. We checked that every HTML form includes the CSRF token from Flask-WTF. We checked that user inputs are validated on both the client side with JavaScript and the server side with WTForms validators. We reviewed variable names across the project to make sure they are consistent and meaningful. We did not find any major issues during the static review

Dynamic Testing:

Dynamic testing means running the system and testing it like a real user would. We opened the website and tested everything manually. We logged in as a student and browsed courses. We enrolled in several courses and watched the seat counts update. We checked the schedule page to make sure the weekly calendar view was correct. We dropped a course and verified the seat count increased. We logged in as an admin and checked that the statistics on the dashboard were accurate. We added a course using the form and verified it appeared in the table. We edited a course and verified the changes were saved. We tried deleting a course with active students and the system blocked it correctly. We added a new student through the form and then logged in with that student's credentials to make sure the account worked. We deleted a student and made sure they could no longer log in. We tested the website on different screen sizes using the browser developer tools. It worked on desktop, tablet, and mobile widths. We tried accessing protected pages directly through the URL without logging in. The system redirected us to the login page each time with an appropriate message. We also left some form fields empty to see how validation handled it. The system showed clear error messages asking us to fill in the required fields. Everything worked as expected during dynamic testing

7. Deployment

We uploaded the full project to GitHub so it can be accessed and reviewed. The repository contains all the source code the README file with setup instructions, and the database file with sample data. Here is the link to the repository:

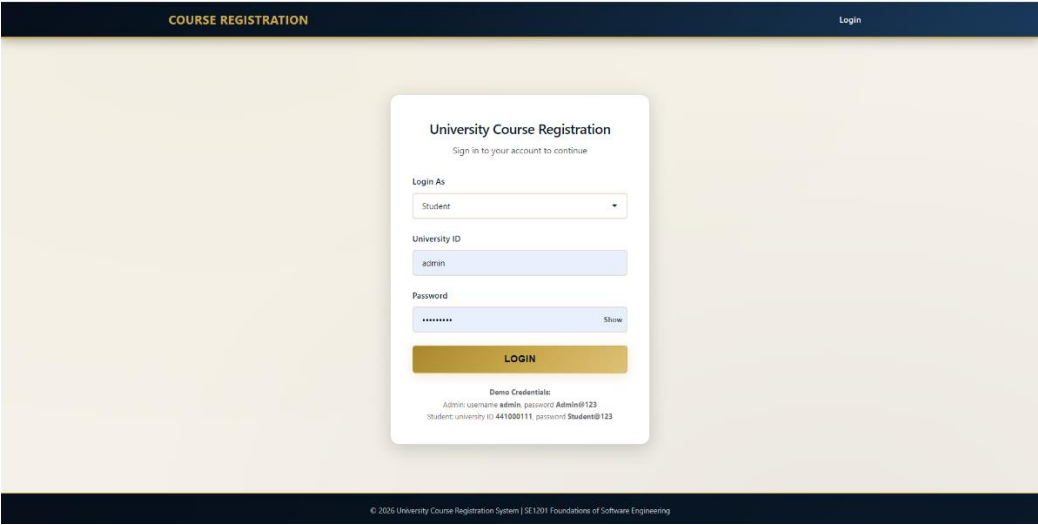
To run the system on another computer, the steps are simple. First, clone the repository from GitHub or extract the project folder. Open a terminal in the project folder. Create a virtual environment by running the command: `python -m venv venv`. Then activate it. On Windows the command is `venv\Scripts\activate`. On Mac or Linux it is `source venv/bin/activate`. Once the virtual environment is active, install the required packages by running: `pip install -r requirements.txt`. After the installation finishes, run the application with: `python app.py`. The server will start and show a message saying it is running on `http://127.0.0.1:5000`. Open a browser and go to that address, then add `/login` to the end. The full address is `http://127.0.0.1:5000/login`

We included sample data in the database so the system is ready to test immediately. There are three student accounts already created. Their university IDs are 441000111, 441000222, and 441000333. All three have the same password which is `Student@123`. There is also an admin account with username `admin` and password `Admin@123`. There are eight courses already in the catalog including CS101 Introduction to Programming, CS201 Data Structures, MATH101 Calculus I, and others. Some courses have prerequisites set up to demonstrate that feature

The system is self-contained and does not need any external services or internet connection to run. Everything works locally on one machine using the SQLite database

Below are screenshots of the running system showing the main pages

1: Login Page



The screenshot shows a web browser displaying the login page of a system titled "COURSE REGISTRATION". The page has a dark blue header with the title on the left and a "Login" link on the right. The main content area is light beige. In the center, there is a white login form titled "University Course Registration" with the subtitle "Sign in to your account to continue". The form includes a "Login As" dropdown menu set to "Student", a "University ID" text input field containing "admin", and a "Password" text input field with masked characters and a "Show" toggle. Below these fields is a prominent orange "LOGIN" button. At the bottom of the form, there is a "Demo Credentials" section listing: "Admin: username admin, password Admin@123" and "Student: university ID 441000111, password Student@123". The footer of the page is dark blue and contains the copyright notice: "© 2025 University Course Registration System | SE101 Foundations of Software Engineering".

2: Admin Dashboard

COURSE REGISTRATION

DashboardManage CoursesView StudentsAdmin: adminLogout

Admin Dashboard

Welcome back, **Administrator!** Manage the system from here.

Admin Access

1

TOTAL STUDENTS

8

TOTAL COURSES

8

TOTAL ENROLLMENTS

1.0

AVG ENROLLMENTS PER COURSE

Manage

COURSES (ADD / EDIT / DELETE)

View

STUDENTS & THEIR ENROLLMENTS

Active

SYSTEM STATUS: RUNNING

Add New

COURSE TO CATALOG

Quick Actions

MANAGE COURSESVIEW STUDENTSLOGOUT

© 2026 University Course Registration System | SE1001: Foundations of Software Engineering

3: Manage Courses Page

COURSE REGISTRATION

DashboardManage CoursesView StudentsAdmin: adminLogout

Manage Courses

Add, edit, or remove courses from the registration catalog.

DISMISSADD NEW COURSE

Course Catalog

8 courses in catalog

#	CODE	COURSE NAME	CREDITS	CAPACITY	ENROLLED	SCHEDULE	PREREQUISITE	ACTIONS
1	CS101	Introduction to Programming	3	28 / 30	1 enrolled	Sunday 10:00 AM - 12:00 PM	None	EDITDELETE
2	CS201	Data Structures	3	24 / 25	0 enrolled	Monday 10:00 AM - 12:00 PM	CS101	EDITDELETE
3	CS301	Database Systems	3	18 / 20	1 enrolled	Thursday 10:00 AM - 12:00 PM	CS201	EDITDELETE
4	ENGL101	English Composition	2	22 / 40	1 enrolled	Monday 10:00 AM - 12:00 PM	None	EDITDELETE
5	MATH101	Calculus I	4	24 / 25	1 enrolled	Sunday 10:00 AM - 12:00 PM	None	EDITDELETE
6	MATH201	Calculus II	4	20 / 20	0 enrolled	Tuesday 10:00 AM - 12:00 PM	MATH101	EDITDELETE
7	PHYS101	Physics I	3	24 / 25	0 enrolled	Wednesday 10:00 AM - 12:00 PM	None	EDITDELETE
8	STAT101	Introduction to Statistics	3	34 / 35	1 enrolled	Wednesday 10:00 AM - 12:00 PM	None	EDITDELETE

© 2026 University Course Registration System | SE1001: Foundations of Software Engineering

4: View Students Page

COURSE REGISTRATION

DashboardManage CoursesView StudentsAdmin: adminLogout

View Students

Overview of all registered students and their current enrollments.

DISMISSMANAGE COURSESADD STUDENT

Total Students

1

TOTAL STUDENTS

Total Enrollments

8

TOTAL ENROLLMENTS

Student Directory

1 student registered

#	UNIVERSITY ID	FULL NAME	EMAIL	MAJOR	COURSE(S)	ACTIONS
1	123456	JONES	jones.john10@gmail.com	MAJ	CS101	EDITDELETE

Enrollment Summary (Students per Course)

This section provides an overview of how many students are enrolled in each course.

CS1017 enrolled

CS2017 enrolled

MATH1017 enrolled

MATH2017 enrolled

PHYS1017 enrolled

ENGL1017 enrolled

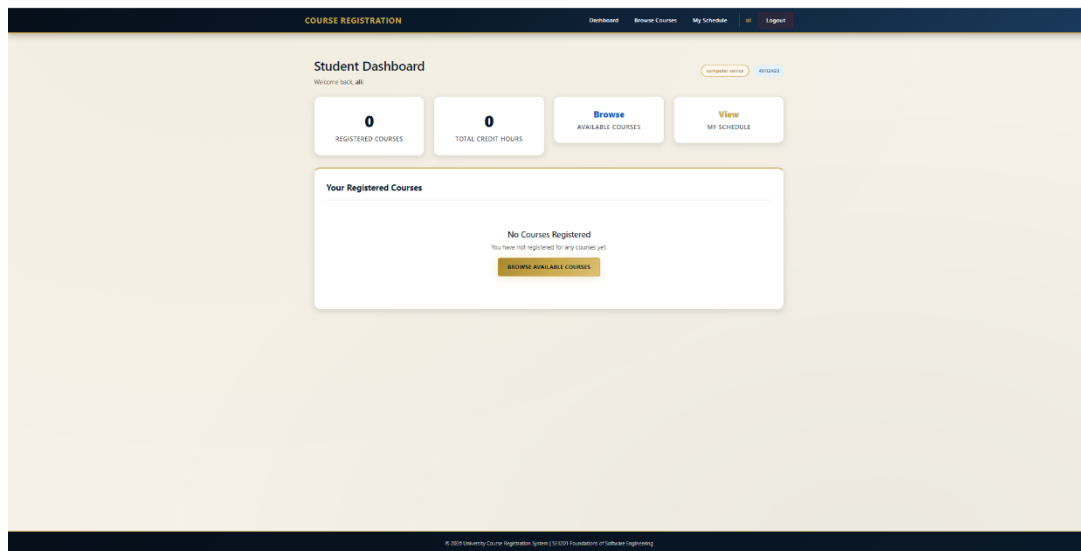
CS3017 enrolled

STAT1017 enrolled

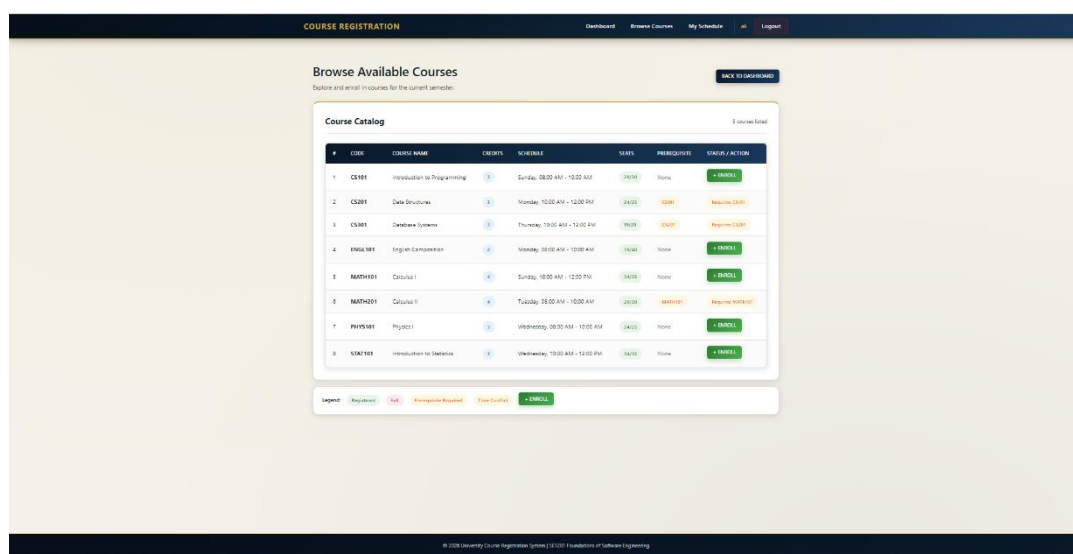
© 2026 University Course Registration System | SE1001: Foundations of Software Engineering

5: Student Dashboard

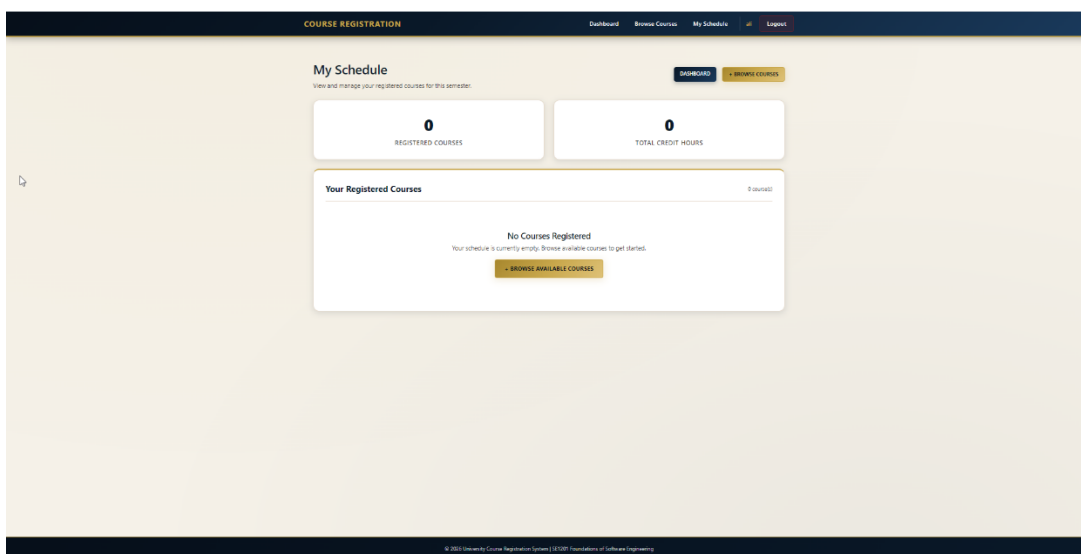
14



6: Browse Courses Page



7: My Schedule Page



These screenshots show the interface and confirm that all the main features are working correctly.

We also prepared the submission folder as required. The folder is named with the team leader's name and version number. Inside it there is a readme.txt file containing the project name, the GitHub link, and all team member names with their student IDs. There is this report file named SE_projectReport. And there is an Excel file named Meeting_logs.xlsx that documents all our meetings

8. Conclusion

We successfully completed the University Course Registration System as required by the SE1201 project. The system allows students to log in, browse courses, enroll with automatic validation, view their weekly schedule, and drop courses. Administrators can log in, manage the course catalog, and manage student accounts. All the features we planned at the beginning were implemented and tested

We followed the System Development Life Cycle throughout the project. We started with a proposal and planning phase. Then we gathered and analyzed requirements. After that we designed the system using six different diagrams. We implemented the code in a modular structure with clear comments. We applied four testing strategies, which were unit testing, integration testing, static testing, and dynamic testing. Finally we deployed the project on GitHub and wrote this report

The main challenges we faced were configuring the Flask blueprints correctly, fixing the CSRF token issues in the templates, and making sure the session could tell the difference between a student and an admin. We solved each of these problems through debugging and research

Looking back, we learned a lot from this project. We now understand why planning and diagrams are important before writing code. We saw how modular code makes a project easier to manage. We learned how testing at different levels helps catch different types of errors. And we gained real experience using Git and GitHub for version controls